

PRACTICE EXERCISES OF THE MICROPROCESSORS & MICROCONTROLLERS

Instructor: The Tung Than

Student's name: Nguyễn Đông Quân

Student code: 24521438

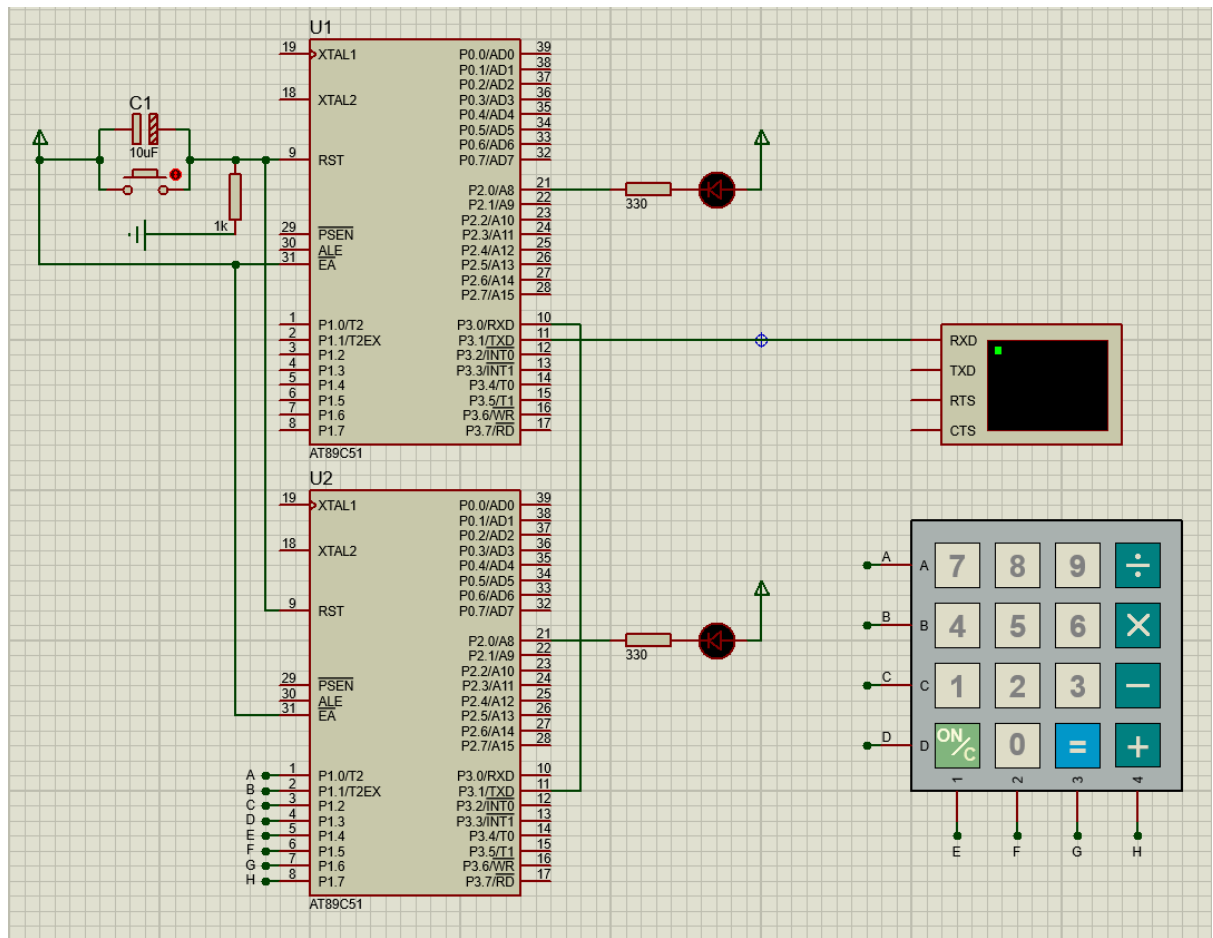
PRACTICE REPORT NO #4:

USING UART

I. Content:

Using AT89C51/AT89C52 to design a handheld calculator, display calculations and results on an LCD that receives data via UART. Split the system into two AT89C51 microcontrollers: the Slave manages the 4×4 keypad and transmits key codes via UART, while the Master receives the data, processes the calculations, and sends the result to the LCD display. Tasks:

- Slave: scan the keypad, encode each key press into 1 byte (0x00– 0x0F), and transmit via UART at 9600 bps.
- Master: receive bytes from the Slave via UART, decode them, and process the arithmetic expression.
- Master sends the result string to the UART LCD using a separate UART channel.
- Use debug LEDs: Slave blinks when transmitting, Master blinks when a byte is successfully received.
- Handle connection loss: 2-second timeout — display "COMM ERR" on the LCD if no data is received.



Mạch tính Master-Slave sử dụng UART

Source code Master:

```

=====
;
; Main.asm file generated by New Project wizard
; Processor: AT89C51
; Compiler: ASEM-51 (Proteus)
=====

$NOMOD51
$INCLUDE (8051.MCU)

=====
; DEFINITIONS & VARIABLES
=====

LED_RX BIT P2.0

flag_ERR BIT 20H.0 ; 1 = Connection timeout
flag_NEW BIT 20H.1 ; 1 = New byte received

```

```
RX_DATA EQU 30H
T_2SEC EQU 31H ; 40 ticks * 50ms = 2s
OP1 EQU 32H
OP2 EQU 33H
OPERATOR EQU 34H
STATE EQU 35H ; 0:OP1, 1:OP, 2:OP2, 3:=
RESULT EQU 36H
```

```
;=====
; RESET and INTERRUPT VECTORS
;=====
```

```
org 0000h
jmp Start

; Timer 0 ISR - 50ms Tick (12 bytes)
org 000Bh
MOV TH0, #4CH
DJNZ T_2SEC, T0_End
SETB flag_ERR
MOV T_2SEC, #40
```

T0_End:

```
RETI
```

```
; UART ISR - Receive Data (11 bytes)
org 0023h
JNB RI, UI_End
CLR RI
CPL LED_RX ; Master blinks on hardware event
MOV RX_DATA, SBUF
SETB flag_NEW
```

UI_End:

```
RETI
```

```
;=====
; CODE SEGMENT
;=====
```

org 0100h

Start:

```
MOV SP, #50H
MOV SCON, #50H ; Mode 1, REN=1
MOV TMOD, #21H ; T1: Mode2, T0: Mode1
MOV TH1, #0FDH ; 9600 bps @ 11.0592 MHz
MOV TL1, #0FDH
SETB TR1
MOV TH0, #4CH
MOV TL0, #00H
SETB TR0
MOV T_2SEC, #40
SETB LED_RX
MOV IE, #92H ; EA, ES, ET0
ACALL Init_Calc
```

Loop:

```
JB flag_ERR, M_Err
JNB flag_NEW, Loop
CLR flag_NEW
MOV T_2SEC, #40
CLR flag_ERR
MOV A, RX_DATA
CJNE A, #0EEH, M_Key ; Process if not heartbeat
jmp Loop
```

M_Key:

```
ACALL Process_Key
jmp Loop
```

M_Err:

```
ACALL Show_Error
jmp Loop
```

; Key processing

Process_Key:

```
MOV DPTR, #Key_Map
MOVC A, @A+DPTR
MOV R4, A
ACALL TX_Char
CJNE A, #'C', PK_Eq
MOV A, #13
ACALL TX_Char
MOV A, #10
ACALL TX_Char
AJMP Init_Calc
```

PK_Eq:

```
CJNE A, #'=', PK_OD
MOV A, STATE
CJNE A, #3, PK_Err
AJMP Calculate_Result
```

PK_OD:

```
CLR C
SUBB A, #'0'
JNC PK_Digit
MOV A, STATE
CJNE A, #1, PK_Err
MOV OPERATOR, R4
MOV STATE, #2
RET
```

PK_Digit:

```
MOV R3, A
SUBB A, #10
JNC PK_Err
MOV A, STATE
JZ PK_S0
CJNE A, #2, PK_Err
MOV OP2, R3
MOV STATE, #3
RET
```

PK_S0:

```
MOV OP1, R3
MOV STATE, #1
RET
```

PK_Err:

```
AJMP Math_Error
```

; Calculation logic

Calculate_Result:

```
MOV A, OPERATOR
CJNE A, #'+', C_Sub
MOV A, OP1
ADD A, OP2
SJMP C_Done
```

C_Sub:

```
CJNE A, #'-', C_Mul
MOV A, OP1
CLR C
SUBB A, OP2
JNC C_Done
MOV A, #'-'
ACALL TX_Char
MOV A, OP2
CLR C
SUBB A, OP1
SJMP C_Done
```

C_Mul:

```
CJNE A, #'*', C_Div
MOV A, OP1
MOV B, OP2
MUL AB
SJMP C_Done
```

C_Div:

```
MOV A, OP2
```

```
JZ PK_Err
MOV A, OP1
MOV B, OP2
DIV AB
```

C_Done:

```
MOV RESULT, A
ACALL Print_Num
AJMP Init_Calc
```

; Subroutines

Show_Error:

```
CLR flag_ERR
MOV DPTR, #Str_Err
ACALL TX_String
AJMP Init_Calc
```

Math_Error:

```
MOV DPTR, #Str_MathErr
ACALL TX_String
AJMP Init_Calc
```

Init_Calc:

```
CLR A
MOV OP1, A
MOV OP2, A
MOV OPERATOR, A
MOV STATE, A
RET
```

Print_Num:

```
MOV A, RESULT
MOV B, #10
DIV AB
JZ P_Units
ADD A, #'0'
ACALL TX_Char
```

P_Units:

```
MOV A, B
ADD A, #'0'
ACALL TX_Char
MOV A, #13
ACALL TX_Char
MOV A, #10
ACALL TX_Char
RET
```

TX_Char:

```
CLR ES ; Prevent RX interrupt during TX
MOV SBUF, A
JNB TI, $
CLR TI
SETB ES
RET
```

TX_String:

```
CLR A
MOVC A, @A+DPTR
JZ TXS_End
ACALL TX_Char
INC DPTR
SJMP TX_String
```

TXS_End:

```
RET
```

; Data tables

Key_Map:

```
DB '7','8','9','/'
DB '4','5','6','*'
DB '1','2','3','- '
DB 'C','0','=','+'
```

Str_Err:

```
DB 13,10,'COMM ERR',13,10,0
```



```
Str_MathErr:
    DB 13,10,'MATH ERR',13,10,0

;=====
END
```

Source code Slave:

```
;=====
; Main.asm file generated by New Project wizard
; Processor: AT89C51
; Compiler: ASEM-51 (Proteus)
;=====

$NOMOD51
$INCLUDE (8051.MCU)

;=====
; DEFINITIONS
;=====

    LED_TX BIT P2.0 ; Blink during UART transmission

;=====
; RESET and INTERRUPT VECTORS
;=====

    org 0000h
    jmp Start

;=====
; CODE SEGMENT
;=====

    org 0100h

Start:
    MOV SP, #50H
```

```
MOV SCON, #50H ; UART mode 1, REN=1
MOV TMOD, #20H ; Timer 1 mode 2
MOV TH1, #0FDH ; 9600 bps @ 11.0592 MHz
MOV TL1, #0FDH
SETB TR1 ; Start Timer 1
SETB LED_TX ; Turn off TX LED
MOV R6, #72 ; Outer loop
MOV R7, #0 ; Inner loop
```

Main_Loop:

```
ACALL Scan_Keypad
CJNE A, #0FFH, Key_Pressed

; Delay for Heartbeat (~1 sec)
DJNZ R7, Main_Loop ; Inner loop: 256 iterations
DJNZ R6, Main_Loop ; Outer loop: 72 iterations

MOV R6, #72 ; Reset heartbeat timer
MOV SBUF, #0EEH ; Send heartbeat byte (0xEE)
ACALL Wait_TX
CPL LED_TX ; Toggle LED to indicate heartbeat TX
SJMP Main_Loop
```

Key_Pressed:

```
MOV R6, #72 ; Reset heartbeat timer upon keypress
MOV R7, #0
CPL LED_TX ; Slave blinks when transmitting
MOV SBUF, A ; Transmit key code via UART
ACALL Wait_TX

ACALL Delay_50ms ; Debounce delay
```

Rel_Wait:

```
ACALL Scan_Keypad
CJNE A, #0FFH, Rel_Wait ; Wait for key release
ACALL Delay_50ms ; Debounce release
SJMP Main_Loop
```

Wait_TX:

```
JNB TI, $ ; Wait until TI flag is set
CLR TI ; Clear TI flag
RET
```

; Scan 4x4 keypad, returns A = index [0..15] or 0xFF

Scan_Keypad:

```
MOV R0, #0 ; R0 = Row index
MOV R1, #0FEH ; R1 = Row selection mask
```

Scan_Row:

```
MOV P1, R1 ; Drive current row low
MOV A, P1 ; Read column data
ANL A, #0F0H ; Mask out row bits
CJNE A, #0F0H, Key_Found ; Key pressed if not 0xF0
INC R0 ; Next row
MOV A, R1
RL A ; Shift 0 to next row
MOV R1, A
CJNE R0, #4, Scan_Row ; Loop 4 times
MOV A, #0FFH ; Return 0xFF
RET
```

Key_Found:

```
SWAP A ; Move cols to low nibble
MOV R2, #0 ; R2 = Column index
```

Col_Scan:

```
RRC A ; Shift right through carry
JNC Got_Col ; Carry 0 = pressed
INC R2
SJMP Col_Scan
```

Got_Col:

```
MOV A, R0
MOV B, #4
MUL AB ; A = row * 4
ADD A, R2 ; A = (row * 4) + col
```

```

RET

; Button debounce
Delay_50ms:
    MOV R4, #100

D1:
    MOV R5, #230
    DJNZ R5, $
    DJNZ R4, D1
    RET

=====
END

```

Giải thích thiết kế:

- **Vì điều khiển:** Sử dụng 2 IC AT89C51 Master-Slave với thạch anh dao động 11.0592 MHz để đảm bảo sai số truyền UART bằng 0%.
- **Cấu trúc Port:**
 - **Port 1 (P1):** Ở U2 (Slave), P1.0 - P1.7 nối trực tiếp với ma trận phím (Keypad 4x4) để quét hàng và cột.
 - **Port 2 (P2):** Chân P2.0 ở cả Master và Slave xuất mức logic 0, nối với LED đỏ qua trở hạn dòng 330 Ohm để báo hiệu trạng thái truyền/nhận.
 - **Port 3 (P3):** Giao tiếp UART, trong đó P3.1 TXD (Slave) truyền dữ liệu sang P3.0 RXD (Master), đồng thời P3.1 TXD (Master) TXD xuất mã ASCII lên Virtual Terminal.
- **Logic điều khiển:**
 - **Master (U1 - Receive & FSM Logic):** Xử lý hoàn toàn bằng ngắt để tối ưu tốc độ.
 - **Nhận dữ liệu:** Ngắt UART tự động lấy dữ liệu từ SBUF mỗi khi cờ RI=1, sau đó lật trạng thái chân P2.0 (CPL LED_RX) trong 50ms để báo hiệu nhận thành công.
 - **FSM:** Quản lý luồng bằng biến STATE. Buộc người dùng nhập đúng trình tự: nhập số thứ nhất → nhập Phép toán → nhập số thứ hai → nhập phím '='. Bất kỳ thao tác sai trình tự nào hoặc chia cho 0 đều báo lỗi MATH ERR.
 - **Timeout:** Ngắt Timer 0 đếm liên tục mỗi 50ms. Nếu quá 2 giây (40 lần tràn) mà không nhận được bất kỳ byte nào từ Slave, Master tự động xóa màn hình và báo lỗi COMM ERR.

▪ **Slave (U2 - Transmit Logic): Quét phím liên tục bằng phương pháp hỏi vòng (Polling).**

- Heartbeat: Nếu không có phím nhấn, tự động gửi mã 0xEE mỗi 1 giây để duy trì kết nối.
- Gửi phím: Nếu có phím nhấn, gửi mã phím qua UART.
- Debug LED: Chân P2.0 tự động đảo trạng thái (CPL LED_TX) ngay trước mỗi lần thực hiện lệnh MOV SBUF, A để báo hiệu đã truyền dữ liệu.

➤ **Tính toán baud rate:**

- **Công thức:** Để đạt baud rate 9600 bps với thạch anh 11.0592 MHz, sử dụng Timer 1 Mode 2. Giá trị nạp cho thanh ghi TH1 được tính bằng công thức:

$$TH1 = 256 - \frac{11.0592 \times 10^6}{12 \times 32 \times 9600} = 256 - 3 = 253 \text{ (0xFD)}$$
- **Cấu hình:** Ở cả Master và Slave, cấu hình UART và Timer được thể hiện qua các lệnh: MOV TH1, #0FDH và MOV TL1, #0FDH.

➤ **Tính toán điện trở sử dụng:** 330 Ohm cho LED đỏ và 999 Ohm cho mạch Reset.

- **Trở 330 Ohm:** Vì $I = \frac{V_{cc} - V_f}{R} = \frac{5 - 2.2}{330} \approx 8.5 \text{ mA} \rightarrow \text{LED sáng an toàn.}$
- **Trở 1k Ohm:** Trở kéo xuống đủ mạnh để thực hiện reset cứng. $I = \frac{V_{cc}}{R} = \frac{5}{1000} = 5 \text{ mA} \rightarrow \text{dòng trong ngưỡng an toàn.}$

Giải thích code Master:

Phần 1: Khai báo cấu hình và ngắt

Khai báo chân tín hiệu LED, các cờ trạng thái (lỗi kết nối, có dữ liệu mới) và cấp phát địa chỉ RAM cho biến (toán hạng, máy trạng thái, bộ đếm). Cấu hình Vector ngắt: Ngắt Timer 0 dùng để đếm Timeout 2 giây (nếu không có dữ liệu sẽ bật cờ lỗi), Ngắt UART dùng để nhận byte, lưu vào bộ đệm và chớp LED phản ứng ngay lập tức.

```

=====
;
; DEFINITIONS & VARIABLES
=====

LED_RX BIT P2.0

flag_ERR BIT 20H.0 ; 1 = Connection timeout
flag_NEW BIT 20H.1 ; 1 = New byte received

RX_DATA EQU 30H
T_2SEC EQU 31H ; 40 ticks * 50ms = 2s
OP1 EQU 32H

```

```

OP2 EQU 33H
OPERATOR EQU 34H
STATE EQU 35H ; 0:OP1, 1:OP, 2:OP2, 3:=
RESULT EQU 36H

;=====
; RESET and INTERRUPT VECTORS
;=====

    org 0000h
    jmp Start

; Timer 0 ISR - 50ms Tick (12 bytes)
    org 000Bh
    MOV TH0, #4CH
    DJNZ T_2SEC, T0_End
    SETB flag_ERR
    MOV T_2SEC, #40
T0_End:
    RETI

; UART ISR - Receive Data (11 bytes)
    org 0023h
    JNB RI, UI_End
    CLR RI
    CPL LED_RX ; Master blinks on hardware event
    MOV RX_DATA, SBUF
    SETB flag_NEW
UI_End:
    RETI

```

Phần 2: Khởi tạo và vòng lặp xử lý chính

Thiết lập tốc độ baud 9600 bps, khởi động Timer và bật ngắt toàn cục. Vòng lặp Loop liên tục kiểm tra trạng thái hệ thống: Nếu mất kết nối (flag_ERR) sẽ báo lỗi; nếu có dữ liệu mới, hệ thống tự động reset bộ đếm Watchdog 2s, lọc bỏ gói tin Heartbeat (0xEE), chỉ đưa mã phím bấm vào bộ xử lý.

```

;=====

```

```
; CODE SEGMENT
```

```
;=====
```

```
org 0100h
```

```
Start:
```

```
MOV SP, #50H
MOV SCON, #50H ; Mode 1, REN=1
MOV TMOD, #21H ; T1: Mode2, T0: Mode1
MOV TH1, #0FDH ; 9600 bps @ 11.0592 MHz
MOV TL1, #0FDH
SETB TR1
MOV TH0, #4CH
MOV TL0, #00H
SETB TR0
MOV T_2SEC, #40
SETB LED_RX
MOV IE, #92H ; EA, ES, ET0
ACALL Init_Calc
```

```
Loop:
```

```
JB flag_ERR, M_Err
JNB flag_NEW, Loop
CLR flag_NEW
MOV T_2SEC, #40
CLR flag_ERR
MOV A, RX_DATA
CJNE A, #0EEH, M_Key ; Process if not heartbeat
jmp Loop
```

```
M_Key:
```

```
ACALL Process_Key
jmp Loop
```

```
M_Err:
```

```
ACALL Show_Error
jmp Loop
```

Phần 3: Xử lý phím

Sử dụng FSM để ràng buộc người dùng nhập đúng cú pháp: Số thứ nhất → Phép toán → Số thứ hai → Dấu '='. Lọc các ký tự bằng mã ASCII, phân biệt chữ số và toán tử. Bất kỳ thao tác sai trình tự nào đều bị điều hướng đến lỗi "MATH ERR".

; Key processing

Process_Key:

```
MOV DPTR, #Key_Map
MOVC A, @A+DPTR
MOV R4, A
ACALL TX_Char
CJNE A, #'C', PK_Eq
MOV A, #13
ACALL TX_Char
MOV A, #10
ACALL TX_Char
AJMP Init_Calc
```

PK_Eq:

```
CJNE A, #'=', PK_OD
MOV A, STATE
CJNE A, #3, PK_Err
AJMP Calculate_Result
```

PK_OD:

```
CLR C
SUBB A, #'0'
JNC PK_Digit
MOV A, STATE
CJNE A, #1, PK_Err
MOV OPERATOR, R4
MOV STATE, #2
RET
```

PK_Digit:

```
MOV R3, A
SUBB A, #10
JNC PK_Err
MOV A, STATE
```



```

JZ PK_S0
CJNE A, #2, PK_Err
MOV OP2, R3
MOV STATE, #3
RET

PK_S0:
MOV OP1, R3
MOV STATE, #1
RET

PK_Err:
AJMP Math_Error

```

Phần 4: Các hàm tính tiện ích và tính toán

Điều hướng ALU thực hiện các phép +, -, *, / cơ bản. Ở phép trừ, nếu số bị trừ nhỏ hơn số trừ (kiểm tra qua cờ Carry), hệ thống tự động in dấu trừ ra LCD trước, sau đó đảo vị trí hai toán hạng (lấy số lớn trừ số bé) để tính ra phần giá trị dương; bắt lỗi chia cho 0. Kết quả được chuyển đổi từ Hex sang ASCII (Print_Num) để xuất ra LCD. Hàm gửi UART (TX_Char) có chốt an toàn tắt ngắt để tránh xung đột màn hình.

```

; Calculation logic
Calculate_Result:
    MOV A, OPERATOR
    CJNE A, #'+', C_Sub
    MOV A, OP1
    ADD A, OP2
    SJMP C_Done

C_Sub:
    CJNE A, #'-', C_Mul
    MOV A, OP1
    CLR C
    SUBB A, OP2
    JNC C_Done
    MOV A, #'-'
    ACALL TX_Char
    MOV A, OP2

```

```
CLR C
SUBB A, OP1
SJMP C_Done
```

C_Mul:

```
CJNE A, #'*', C_Div
MOV A, OP1
MOV B, OP2
MUL AB
SJMP C_Done
```

C_Div:

```
MOV A, OP2
JZ PK_Err
MOV A, OP1
MOV B, OP2
DIV AB
```

C_Done:

```
MOV RESULT, A
ACALL Print_Num
AJMP Init_Calc
```

; Subroutines

Show_Error:

```
CLR flag_ERR
MOV DPTR, #Str_Err
ACALL TX_String
AJMP Init_Calc
```

Math_Error:

```
MOV DPTR, #Str_MathErr
ACALL TX_String
AJMP Init_Calc
```

Init_Calc:

```
CLR A
MOV OP1, A
```

```
MOV OP2, A
MOV OPERATOR, A
MOV STATE, A
RET
```

Print_Num:

```
MOV A, RESULT
MOV B, #10
DIV AB
JZ P_Units
ADD A, #'0'
ACALL TX_Char
```

P_Units:

```
MOV A, B
ADD A, #'0'
ACALL TX_Char
MOV A, #13
ACALL TX_Char
MOV A, #10
ACALL TX_Char
RET
```

TX_Char:

```
CLR ES ; Prevent RX interrupt during TX
MOV SBUF, A
JNB TI, $
CLR TI
SETB ES
RET
```

TX_String:

```
CLR A
MOVC A, @A+DPTR
JZ TXS_End
ACALL TX_Char
INC DPTR
SJMP TX_String
```

```

TXS_End:
    RET

; Data tables
Key_Map:
    DB '7','8','9','/'
    DB '4','5','6','*'
    DB '1','2','3','-'
    DB 'C','0','=','+'

Str_Err:
    DB 13,10,'COMM ERR',13,10,0

Str_MathErr:
    DB 13,10,'MATH ERR',13,10,0

;=====
END

```

Giải thích code Slave:

Phần 1: Khai báo cấu hình và ngắt

Khai báo chân LED phát tín hiệu và cấu hình tốc độ Baud. Chương trình sử dụng kỹ thuật Polling: Liên tục quét phím, nếu rảnh rồi (không có phím) quá 1 giây thì tự động xuất lệnh chớp LED (CPL LED_TX) và gửi mã Heartbeat (0xEE). Nếu có phím, ngay lập tức gửi mã đi, nháy LED, và chờ nhả phím với cơ chế debounce (Delay_50ms).

```

;=====
; DEFINITIONS
;=====

    LED_TX BIT P2.0 ; Blink during UART transmission

;=====
; RESET and INTERRUPT VECTORS
;=====

    org 0000h
    jmp Start

```

```
=====
;
; CODE SEGMENT
;
=====
```

```
org 0100h
```

Start:

```
MOV SP, #50H
MOV SCON, #50H ; UART mode 1, REN=1
MOV TMOD, #20H ; Timer 1 mode 2
MOV TH1, #0FDH ; 9600 bps @ 11.0592 MHz
MOV TL1, #0FDH
SETB TR1 ; Start Timer 1
SETB LED_TX ; Turn off TX LED
MOV R6, #72 ; Outer loop
MOV R7, #0 ; Inner loop
```

Main_Loop:

```
ACALL Scan_Keypad
CJNE A, #0FFH, Key_Pressed

; Delay for Heartbeat (~1 sec)
DJNZ R7, Main_Loop ; Inner loop: 256 iterations
DJNZ R6, Main_Loop ; Outer loop: 72 iterations

MOV R6, #72 ; Reset heartbeat timer
MOV SBUF, #0EEH ; Send heartbeat byte (0xEE)
ACALL Wait_TX
CPL LED_TX ; Toggle LED to indicate heartbeat TX
SJMP Main_Loop
```

Key_Pressed:

```
MOV R6, #72 ; Reset heartbeat timer upon keypress
MOV R7, #0
CPL LED_TX ; Slave blinks when transmitting
MOV SBUF, A ; Transmit key code via UART
ACALL Wait_TX
```

```
ACALL Delay_50ms ; Debounce delay
```

Rel_Wait:

```
ACALL Scan_Keypad  
CJNE A, #0FFH, Rel_Wait ; Wait for key release  
ACALL Delay_50ms ; Debounce release  
SJMP Main_Loop
```

Wait_TX:

```
JNB TI, $ ; Wait until TI flag is set  
CLR TI ; Clear TI flag  
RET
```

Phần 2: Quét ma trận phím và debounce

Thuật toán quét ma trận 4x4: Dịch dần mức logic 0 qua 4 hàng để kiểm tra từng cột (ANL A, #0F0H). Khi phát hiện nút được nhấn, sử dụng phép tính toán học (Row * 4) + Column để nội suy ra vị trí Index chuẩn xác từ 0 đến 15. Hàm Delay_50ms được tái sử dụng để làm vòng lặp tiêu tốn thời gian chống nhiễu phần cứng.

```
; Scan 4x4 keypad, returns A = index [0..15] or 0xFF
```

Scan_Keypad:

```
MOV R0, #0 ; R0 = Row index  
MOV R1, #0FEH ; R1 = Row selection mask
```

Scan_Row:

```
MOV P1, R1 ; Drive current row low  
MOV A, P1 ; Read column data  
ANL A, #0F0H ; Mask out row bits  
CJNE A, #0F0H, Key_Found ; Key pressed if not 0xF0  
INC R0 ; Next row  
MOV A, R1  
RL A ; Shift 0 to next row  
MOV R1, A  
CJNE R0, #4, Scan_Row ; Loop 4 times  
MOV A, #0FFH ; Return 0xFF  
RET
```

Key_Found:

```

        SWAP A ; Move cols to low nibble
        MOV R2, #0 ; R2 = Column index

Col_Scan:
        RRC A ; Shift right through carry
        JNC Got_Col ; Carry 0 = pressed
        INC R2
        SJMP Col_Scan

Got_Col:
        MOV A, R0
        MOV B, #4
        MUL AB ; A = row * 4
        ADD A, R2 ; A = (row * 4) + col
        RET

; Button debounce
Delay_50ms:
        MOV R4, #100

D1:
        MOV R5, #230
        DJNZ R5, $
        DJNZ R4, D1
        RET

=====
        END

```

II. References

- TS. Vũ Đức Lung, TS. Lê Quang Minh, ThS. Phan Đình Duy, *Giáo trình Vi điều khiển*, Trường Đại học Công nghệ Thông tin - ĐHQG TP.HCM, 2015
- Link video:
https://drive.google.com/file/d/16kVrws4c_aH7iKyCIJO9c0kidGd0w-y/view?usp=sharing